# Руководство по интеграции платформы Cloudhike и настройке аутентификации GitHub в проекте DevCrew

В архитектуре платформы **DevCrew** надежная работа автономных ИИ-агентов (*Analyst, Developer, Reviewer, Tester*) опирается на два ключевых фундаментальных сервиса: облачную платформу **Cloudhike**, выступающую комплексным all-in-one бэкендом, и **двухуровневую авторизацию GitHub**, изолирующую сессии людей от программных прав агентов.

Ниже представлено полное руководство с архитектурным описанием, пошаговыми инструкциями и примерами программного кода.

---

## 1. Архитектурная роль платформы Cloudhike в DevCrew

В традиционной веб-разработке создание сложного приложения требует ручного связывания множества разрозненных провайдеров (отдельно управляемый Postgres, внешний auth-провайдер, AWS S3 и очереди фоновых воркеров). **Cloudhike (cloudhike.dev)** решает проблему инфраструктурной фрагментации, предоставляя разработчику все необходимые серверные сервисы в одном окне с развертыванием в один клик.

### 1.1. Для чего используется Cloudhike и его сервисы в DevCrew

Проект использует четыре ключевых сервиса платформы:

1. **Postgres Database (Управляемая база данных PostgreSQL 14+)**:
   - **Для чего**: Единый источник правды для всей системы. Хранит учетные записи пользователей, воркспейсы, задачи канбан-доски, настройки системных промптов и навыков агентов, лимиты максимального числа итераций (*max turns*), финансовые бюджеты и начисленные затраты на токены, а также ссылки на открытые pull request.
   - **Интеграция**: Приложение подключается к базе через **Drizzle ORM** ( `db/schema.ts` ).

2. **Hosted Better Auth (Облачная аутентификация)**:
   - **Для чего**: Управление сессиями, валидация cookie-токенов, безопасный вход по email/паролю, Google и GitHub без необходимости писать и поддерживать собственный сервер авторизации.
   - **Особенность**: Функция *«Store user in project database»* автоматически создает строку в таблице `users` при первом успешном входе пользователя без необходимости сложных вебхуков.

3. **Cloudhike Jobs (Распределенные очереди фоновых задач)**:
   - **Для чего**: Оркестрация длительных асинхронных операций. Пошаговый запуск цепочки субагентов ( `/api/jobs/task-run` ) и аудит безопасности Switch Trust ( `/api/jobs/switch-scan` ) могут занимать от десятков секунд до нескольких минут. Cloudhike Jobs запускает их в фоне через подписанные вебхуки ( `x-cloudhike-signature` ), исключая таймауты и блокировку пользовательского интерфейса.

4. **Cloudhike Storage (S3-совместимое объектное хранилище)**:
   - **Для чего**: Безопасная загрузка и отдача пользовательских медиафайлов и приватных логотипов рабочих пространств без необходимости ручной настройки политик доступа в консоли AWS IAM.

## 1.2. Примеры программного подключения сервисов Cloudhike

1. Подключение к PostgreSQL через Drizzle ORM ( `db/index.ts` )

```
import { drizzle } from "drizzle-orm/node-postgres";
import { Pool } from "pg";
import * as schema from "./schema";

const connectionString = process.env.DATABASE_URL;

if (!connectionString) {
  throw new Error("Переменная окружения DATABASE_URL не задана в файле .env");
}

const pool = new Pool({
  connectionString,
  ssl: process.env.NODE_ENV === "production" ? { rejectUnauthorized: false } : false,
});

export const db = drizzle(pool, { schema });
```

Синхронизация схемы таблиц `db/schema.ts` с базой данных Cloudhike выполняется командой:

```
npm run db:push
```

2. Проверка сессии в Hosted Better Auth ( `lib/auth.ts` )

```
import { cookies } from "next/headers";
import { db } from "@/db";
import { users } from "@/db/schema";
import { eq } from "drizzle-orm";

export async function getCurrentUser() {
  const authUrl = process.env.AUTH_URL;
  const publishableKey = process.env.AUTH_PUBLISHABLE_KEY;

  if (!authUrl || !publishableKey) {
    throw new Error("AUTH_URL или AUTH_PUBLISHABLE_KEY отсутствуют в .env");
  }

  const cookieStore = await cookies();
  const sessionToken = cookieStore.get("better_auth_session")?.value;

  if (!sessionToken) return null;

  // Запрос верификации сессионного токена в сервисе Cloudhike Auth
  const response = await fetch(`${authUrl}/api/auth/get-session`, {
    headers: {
      Authorization: `Bearer ${sessionToken}`,
      "x-publishable-key": publishableKey,
    },
    cache: "no-store",
```

```
  });

  if (!response.ok) return null;

  const data = await response.json();
  return data.user || null;
}
```

3. Серверный клиент Cloudhike Jobs (`lib/jobs.ts`)

В продакшене запросы отправляются в Cloudhike Jobs API. При локальной разработке без заданных ключей срабатывает фоновый fallback Next.js (`after()`):

```
interface EnqueueJobParams {
  endpoint: string;
  payload: Record<string, any>;
}

export async function enqueueCloudhikeJob({ endpoint, payload }: EnqueueJobParams) {
  const jobsApiUrl = process.env.JOBS_API_URL;
  const jobsApiKey = process.env.JOBS_API_KEY;
  const appUrl = process.env.NEXT_PUBLIC_APP_URL;

  // Локальный режим разработки: если очереди не сконфигурированы,
  // выполнение запускается в фоновом режиме сервера Next.js
  if (!jobsApiUrl || !jobsApiKey) {
    console.log(`[Jobs Local Fallback] Асинхронный запуск задачи на ${endpoint}`);
    return { id: "local-job-" + Date.now(), status: "queued" };
  }

  const targetUrl = `${appUrl}${endpoint}`;
  const response = await fetch(`${jobsApiUrl}/runs`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-api-key": jobsApiKey,
    },
    body: JSON.stringify({
      target_url: targetUrl,
      payload,
      max_attempts: 3,
    }),
  });

  if (!response.ok) {
    const errorText = await response.text();
    throw new Error(`Ошибка постановки задачи в Cloudhike Jobs: ${response.status}
${errorText}`);
  }

  return await response.json();
}
```

4. Загрузка логотипов в Cloudhike Storage (`lib/storage.ts`)

```
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";

const s3Client = new S3Client({
  endpoint: process.env.S3_ENDPOINT,
  region: process.env.S3_REGION || "us-east-1",
  credentials: {
    accessKeyId: process.env.S3_ACCESS_KEY || "",
    secretAccessKey: process.env.S3_SECRET_KEY || "",
  },
  forcePathStyle: true,
});

export async function uploadWorkspaceLogo(
  workspaceId: string,
  fileBuffer: Buffer,
  contentType: string
): Promise<string> {
  const bucketName = process.env.S3_BUCKET || "workspace";
  const fileKey = `logos/${workspaceId}-${Date.now()}.png`;

  await s3Client.send(
    new PutObjectCommand({
      Bucket: bucketName,
      Key: fileKey,
      Body: fileBuffer,
      ContentType: contentType,
    })
  );

  return `/api/workspaces/${workspaceId}/logo?key=${encodeURIComponent(fileKey)}`;
}
```

---

## 2. Архитектура и настройка аутентификации GitHub

В платформе DevCrew авторизация через GitHub разделена на **два независимых уровня**:

- **GitHub Sign-In (Вход пользователя в систему)**: Служит исключительно для идентификации человека на странице `/sign-in`. Приложение не запрашивает прав на модификацию репозиториев от имени вошедшего пользователя.
- **GitHub OAuth App (Доступ ИИ-агентов к репозиторию)**: Выделенное OAuth-приложение, через которое конкретное рабочее пространство (`workspace`) получает права на чтение issues, создание изолированных веток, коммиты файлов и открытие pull request. Токены сохраняются в зашифрованном виде.

---

### 2.1. Пошаговая инструкция по настройке GitHub OAuth App

1. **Создание приложения в GitHub**:
   - Перейдите в github.com/settings/developers → **OAuth Apps** → **New OAuth App**.
   - **Application name**: `DevCrew AI Agent Platform`

- **Homepage URL**: базовый адрес приложения (для локальной разработки: `http://localhost:3000`, значение `NEXT_PUBLIC_APP_URL`).
- **Authorization callback URL**: строгий адрес обработчика: `<NEXT_PUBLIC_APP_URL>/api/github/callback` (например, `http://localhost:3000/api/github/callback`).
- Нажмите **Register application**.

2. **Получение Client ID и генерация Client Secret**:
   - Скопируйте сгенерированный **Client ID**.
   - Нажмите кнопку **Generate a new client secret** и скопируйте появившийся секретный ключ.

3. **Генерация ключа шифрования токенов (TOKEN_ENCRYPTION_KEY)**:
   - Для безопасного хранения полученных токенов в базе данных сгенерируйте надежный случайный ключ в терминале:

```
openssl rand -base64 48
```

1. **Заполнение файла .env**:

```
GITHUB_CLIENT_ID="ваш_github_client_id"
GITHUB_CLIENT_SECRET="ваш_github_client_secret"
TOKEN_ENCRYPTION_KEY="ваш_сгенерированный_base64_ключ"
```

*Важное правило*: Одно OAuth App в GitHub поддерживает только один Callback URL. Поэтому для `local`, `staging` и `production` создаются отдельные приложения в GitHub Developers Settings.

---

## 2.2. Программная реализация подключения GitHub в кодовой базе

1. Модуль шифрования токенов AES-256-GCM (`lib/crypto.ts`)

Токены доступа агентов шифруются перед записью в базу данных:

```
import crypto from "crypto";

const ALGORITHM = "aes-256-gcm";

export function encryptToken(plainText: string, encryptionKey: string): string {
  const key = crypto.createHash("sha256").update(encryptionKey).digest();
  const iv = crypto.randomBytes(16);
  const cipher = crypto.createCipheriv(ALGORITHM, key, iv);

  const encrypted = Buffer.concat([cipher.update(plainText, "utf8"), cipher.final()]);
  const tag = cipher.getAuthTag();

  return `${iv.toString("hex")}:${tag.toString("hex")}:${encrypted.toString("hex")}`;
}

export function decryptToken(encryptedPayload: string, encryptionKey: string): string {
  const key = crypto.createHash("sha256").update(encryptionKey).digest();
  const [ivHex, tagHex, encryptedHex] = encryptedPayload.split(":");
```

```
  if (!ivHex || !tagHex || !encryptedHex) {
    throw new Error("Некорректный формат зашифрованных данных.");
  }

  const iv = Buffer.from(ivHex, "hex");
  const tag = Buffer.from(tagHex, "hex");
  const encryptedText = Buffer.from(encryptedHex, "hex");

  const decipher = crypto.createDecipheriv(ALGORITHM, key, iv);
  decipher.setAuthTag(tag);

  const decrypted = Buffer.concat([decipher.update(encryptedText), decipher.final()]);
  return decrypted.toString("utf8");
}
```

2. Маршрут инициации подключения ( `app/api/github/connect/route.ts` )

```
import { NextResponse } from "next/server";

export async function GET() {
  const clientId = process.env.GITHUB_CLIENT_ID;
  const appUrl = process.env.NEXT_PUBLIC_APP_URL;

  const redirectUri = `${appUrl}/api/github/callback`;
  const scopes = ["repo", "read:user"].join(" ");

  const githubAuthUrl = `[suspicious link removed]
{clientId}&redirect_uri=${encodeURIComponent(
    redirectUri
  )}&scope=${encodeURIComponent(scopes)}`;

  return NextResponse.redirect(githubAuthUrl);
}
```

3. Маршрут обработки Callback ( `app/api/github/callback/route.ts` )

```
import { NextRequest, NextResponse } from "next/server";
import { encryptToken } from "@/lib/crypto";

export async function GET(request: NextRequest) {
  const searchParams = request.nextUrl.searchParams;
  const code = searchParams.get("code");
  const appUrl = process.env.NEXT_PUBLIC_APP_URL || "http://localhost:3000";

  if (!code) {
    return NextResponse.redirect(`${appUrl}/workspaces/new?error=missing_code`);
  }

  // Обмен временного code на постоянный access token
  const tokenResponse = await fetch("[suspicious link removed]", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Accept: "application/json",
```

```
    },
    body: JSON.stringify({
      client_id: process.env.GITHUB_CLIENT_ID,
      client_secret: process.env.GITHUB_CLIENT_SECRET,
      code,
    }),
  });

  const tokenData = await tokenResponse.json();
  const encryptionKey = process.env.TOKEN_ENCRYPTION_KEY;

  if (!encryptionKey || !tokenData.access_token) {
    return NextResponse.redirect(`${appUrl}/workspaces/new?error=auth_failed`);
  }

  // Шифрование токена перед сохранением в сессии создания воркспейса
  const encryptedToken = encryptToken(tokenData.access_token, encryptionKey);

  const response = NextResponse.redirect(`${appUrl}/workspaces/new?connected=true`);
  response.cookies.set("devcrew_github_token", encryptedToken, {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    path: "/",
    maxAge: 3600,
  });

  return response;
}
```

4. Фабрика клиента Octokit для агентов ( `lib/github.ts` )

```
import { Octokit } from "@octokit/rest";
import { decryptToken } from "@/lib/crypto";

export async function createGitHubClientForWorkspace(encryptedToken: string):
Promise<Octokit> {
  const encryptionKey = process.env.TOKEN_ENCRYPTION_KEY;

  if (!encryptionKey) {
    throw new Error("TOKEN_ENCRYPTION_KEY отсутствует в конфигурации окружения.");
  }

  const plainAccessToken = decryptToken(encryptedToken, encryptionKey);

  return new Octokit({
    auth: plainAccessToken,
  });
}
```

---

## 2.3. Настройка GitHub Sign-In через панель Cloudhike Auth

Вход пользователей по кнопке «Войти через GitHub» настраивается в облачной консоли Cloudhike:

1. Перейдите в дашборд проекта на **cloudhike.dev** → раздел **Auth (Authentication)** → вкладка **Providers**.
2. Найдите провайдер **GitHub** и переведите его в положение **On**.
3. Для быстрой разработки включите **Use Cloudhike shared credentials** (в продакшене можно указать собственный Client ID и Secret для входа).
4. Укажите Redirect URL (`http://localhost:3000` для локальной разработки).
5. Убедитесь, что активна опция **Store user in project database** (для сохранения пользователей в таблицу `users`).
6. Нажмите **Save providers**.

---

## 3. Сводная спецификация переменных окружения ( `.env` )

Для корректной работы всех модулей файл `.env` должен содержать следующие ключи:

```
# Базовый адрес приложения (должен строго совпадать с Homepage URL в GitHub OAuth App)
NEXT_PUBLIC_APP_URL="http://localhost:3000"

# Cloudhike: PostgreSQL (Drizzle ORM)
DATABASE_URL="postgresql://postgres:password@db.cloudhike.dev:5432/devcrew?
sslmode=require"

# Cloudhike: Hosted Better Auth (указывайте финальный хост без www-редиректов!)
AUTH_URL="https://auth.cloudhike.dev"
AUTH_PUBLISHABLE_KEY="pk_live_xxxxxxxxxxxxxxx"
AUTH_SECRET_KEY="sk_live_xxxxxxxxxxxxxxx"

# Cloudhike: Фоновые задачи (Cloudhike Jobs)
JOBS_API_URL="[suspicious link removed]"
JOBS_API_KEY="jobs_key_xxxxxxxxxxxxxxx"

# Cloudhike: S3 Storage (для пользовательских логотипов)
S3_ENDPOINT="https://storage.cloudhike.dev"
S3_BUCKET="workspace-assets"
S3_ACCESS_KEY="s3_access_xxxxxxxx"
S3_SECRET_KEY="s3_secret_xxxxxxxx"
S3_REGION="us-east-1"

# GitHub OAuth App (доступ агентов к коду и репозиториям)
GITHUB_CLIENT_ID="Iv1.xxxxxxxxxxxxxxx"
GITHUB_CLIENT_SECRET="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
TOKEN_ENCRYPTION_KEY="случайный_ключ_из_openssl_rand_base64_48"

# OpenAI API (исполнительный контур агентов)
OPENAI_API_KEY="sk-proj-xxxxxxxxxxxxxxxxxxxxxxx"
```

### Важные нюансы локальной отладки:

- **Устранение потери заголовков при редиректах**: Если вход по email работает, а вход через GitHub выдает ошибку *«We couldn't complete sign-in»*, проверьте переменную `AUTH_URL`. Если домен перенаправляет

трафик (например, `cloudhike.dev` → `www.cloudhike.dev`), функция `fetch` в Node.js автоматически сбрасывает заголовок `Authorization`. Указывайте конечный домен.

- **Тестирование фоновых вебхуков через ngrok**: Для локальной отладки очередей Cloudhike Jobs и сканирования Switch Trust запустите туннель:

```
ngrok http 3000
```

Затем укажите полученный HTTPS-адрес ngrok в `NEXT_PUBLIC_APP_URL` и обновите Callback URL в настройках приложения GitHub OAuth.